

SkillSwap - Documentazione Backend

Panoramica

SkillSwap e' un'applicazione per lo scambio di competenze tra utenti. Il backend e' un'API REST scritta con FastAPI (Python) che usa SQLite come database e JWT per l'autenticazione.

Architettura

Componente	File	Ruolo
Database	app/database.py	Connessione SQLite con SQLAlchemy
Tabelle	app/db_models.py	5 tabelle: users, skills, user_skills, session_requests, feedback
Schemi API	app/models.py	Validazione dati con Pydantic
Router	app/routers/	7 gruppi di endpoint

Logica per dominio

1. Autenticazione (/auth) - auth.py

Registrazione: valida email/password, hash la password con bcrypt, salva l'utente, restituisce un JWT (firmato con chiave segreta + scadenza 24h).

Login: verifica password con bcrypt, restituisce JWT.

Profilo personale (/auth/me): decodifica il JWT dall'header 'Authorization: Bearer <token>', restituisce dati + skill dell'utente.

Modifica profilo (/auth/profile): aggiorna solo i campi inviati (patch logica).

2. Skill (/skills) - skills.py

Catalogo globale: GET /skills/ elenca tutte le skill esistenti.

Creazione skill: POST /skills/ con controllo duplicati (409 se gia' esiste).

Skill personali: CRUD completo su /skills/my/. Se aggiungi una skill che non esiste nel catalogo, viene creata automaticamente. Non puoi avere due volte la stessa skill con lo stesso tipo (offered/wanted).

3. Utenti e Ricerca (/users) - users.py

Profilo pubblico (/users/public/{id}): nome, bio, citta', media voti calcolata dai feedback.

Skill pubbliche (/users/{id}/skills): divide in offered_skills e wanted_skills.

Ricerca utenti (/users/search?q=...): cerca per nome, bio o nome skill (con ILIKE), con paginazione. Se sei loggato, calcola il match reciproco: io offro X che tu cerchi E tu offri Y che io cerco.

Statistiche globali (/users/stats): conta utenti, skill, associazioni.

4. Richieste di Sessione (/requests) - request.py

Invio richiesta: POST /requests/ - un utente chiede una sessione a un altro. Controlli: non a se stessi, destinatario e skill devono esistere.

Macchina a stati delle richieste:

Stato	Azione	Chi puo' farlo
pending -> accepted	accept	Solo il destinatario
pending -> rejected	reject	Solo il destinatario

SkillSwap - Documentazione Backend

pending -> cancelled	cancel	Solo il mittente
accepted -> completed	confirm_completion	Entrambi (doppia conferma)
accepted -> cancelled	cancel	Solo il mittente

La logica a doppia conferma: la sessione passa a 'completed' solo quando sia mittente che destinatario hanno chiamato confirm_completion.

5. Feedback (/feedback) - feedback.py

Solo sessioni completate: puoi lasciare feedback (voto 1-5 + commento) solo se la sessione e' in stato 'completed'.

Solo partecipanti: devi essere mittente o destinatario della sessione.

Un feedback per sessione: se hai gia' valutato, errore 409.

Media voti calcolata su tutte le sessioni a cui l'utente ha partecipato.

6. Reports & Blocks (/reports, /blocks) - stub

Endpoint creati ma non implementati: restituiscono sempre un messaggio fittizio.

Flusso tipico completo

1. Registrazione / Login -> ottieni JWT
2. Aggiungi skill al profilo (offered / wanted)
3. Cerca utenti -> vedi match reciproci
4. Invia richiesta di sessione
5. Destinatario accetta
6. Dopo la sessione, entrambi confermano il completamento
7. Lascia feedback (voto + commento)

Il backend e' in versione 0.2.0 - funzionale ma con parti ancora in sviluppo (reports, blocks).

Database: SQLite (skillswap.db) | Framework: FastAPI | Autenticazione: JWT + bcrypt